

CANARA ENGINEERING COLLEGE

Benjanapadavu, Bantwal Taluk, Mangaluru, D.K. – 574219
(Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

COURSE PROJECT ASSIGNMENT REPORT

CRYPTOGRAPHY & NETWORK SECURITY (BCS703) — SEMESTER VII (B.E. CSE)

KERBEROS AS EXCHANGE SIMULATION

An Educational Full-Stack Software Simulation of Initial Single Sign-On Authentication

PROJECT IDENTIFICATION

Project Group:	G17
Project Number:	P17
Course / Code:	CNS / BCS703
Academic Year:	2025 – 2026
Software Version:	v2.0 (FastAPI + React)

STUDENT DETAILS & CONTRIBUTIONS

4CB23CS161 Member 1	Backend Architecture, Cryptographic Engineering, Testing, KDC Service Engine
4CB23CS160 Member 2	Frontend Architecture, Glassmorphic UI, Protocol Telemetry, API Integration

Academic Scope & Disclaimer: This course project report documents an educational software simulation developed for the course Cryptography & Network Security (BCS703). The simulation demonstrates the initial Authentication Server (AS) exchange of the Kerberos V5 protocol (RFC 4120) using modern REST/JSON transport and standard cryptography (PBKDF2-HMAC-SHA256, AES-256-GCM). It is engineered for transparent classroom visualization and does not claim RFC 4120 binary ASN.1 wire compatibility.

Course Coordinator

Dept. of Computer Science & Engg.

Head of Department (HOD)

Dept. of Computer Science & Engg.

1 Description of Requirements

1.1 Introduction to Kerberos & The Authentication Server (AS) Exchange

Kerberos is a centralized, trusted third-party network authentication protocol originated at MIT (RFC 4120). Designed around the classical "three-headed dog" architectural model, Kerberos establishes mutual authentication across three entities: the **Client**, the **Key Distribution Center (KDC)**, and the **Application Server**. The KDC houses two functional servers: the Authentication Server (AS) and the Ticket Granting Server (TGS). The **Authentication Server (AS) Exchange** constitutes Phase 1 of the Kerberos protocol—the foundational gateway where a client proves its identity without ever transmitting passwords across the network, thereby obtaining a Ticket Granting Ticket (TGT) and an ephemeral session key ($K_{C,TGS}$).

1.2 Problem Statement & Motivation

Traditional distributed login mechanisms transmit credentials or static hashes over insecure channels, exposing networks to eavesdropping, packet sniffing, pass-the-hash attacks, and password replay. Enterprise Kerberos implementations (e.g., Microsoft Active Directory, MIT Kerberos) provide robust defense but operate as opaque binary black boxes over binary ASN.1 DER streams on UDP/TCP port 88. For undergraduate computer science students, tracing individual keys, timestamps, nonces, and ticket structures inside these production streams is exceedingly difficult. **Motivation:** To design and implement a full-stack, transparent, and cryptographically verified educational simulation that visualizes every step of the AS exchange in real time.

1.3 Project Objectives

- **Protocol Mechanics:** Implement standard RFC 4120 Authentication Server semantics (``KRB_AS_REQ`` and ``KRB_AS_REP``).
- **Cryptographic Grounding:** Integrate PBKDF2-HMAC-SHA256 (100,000 iterations) for string-to-key derivation and authenticated AES-256-GCM with unique 96-bit random IVs for ticket confidentiality and tamper-evident integrity.
- **Replay & Request Correlation:** Enforce client nonces (N_1) for request-response matching and server-side UTC clock-skew validation ($\pm 300\text{s}$).
- **Interactive Visual Telemetry:** Construct a responsive React dashboard demonstrating 8-step live telemetry progression.
- **Formal Verification:** Provide automated test coverage across frontend and backend tiers (21 verified automated tests).

1.4 Functional & Non-Functional Requirements

Req ID	Functional Requirement (FR)	Req ID	Non-Functional Requirement (NFR)
FR-1	Client request generation with principal, realm, service, nonce, timestamp, and lifetime.	NFR-1	Confidentiality: Zero plain passwords, salts, or master keys in API JSON payloads.
FR-2	Server-side validation of realm, service, and clock skew freshness ($\leq 300\text{s}$).	NFR-2	Integrity: AES-GCM 128-bit authentication tags; tamper attempts throw <code>`InvalidTag`</code> .
FR-3	KDC user lookup and client key derivation (K_C) via PBKDF2; error mapping if unknown.	NFR-3	Replay Defense: Multi-factor protection via nonces, timestamps, and ticket lifetimes.
FR-4	Plaintext TGT assembly and encryption using master key K_{TGS} via AES-256-GCM.	NFR-4	Performance: Sub-50ms server response latency for cryptographic processing.
FR-5	Client package construction and encryption using K_C with session key and nonce echo.	NFR-5	Testability: Fully automated unit and integration suites with 100% pass rates.
FR-6	Structured RFC error emission (<code>`KDC_ERR_C_PRINCIPAL_UNKNOWN`</code> , <code>`KRB_AP_ERR_SKEW`</code>).		

1.5 System Environment & Project Limitations

Environment: Python 3.12, FastAPI, Pydantic V2, PyCA Cryptography, Node.js v18+, React 18, Vite, Vitest, Pytest.
Limitations: Focus is confined to the initial AS Exchange (Phase 1). Subsequent TGS service ticket exchanges (Phase 2) and AP mutual authentication (Phase 3) constitute future extensions. Transport utilizes JSON/REST over HTTP rather than raw ASN.1 over UDP port 88.

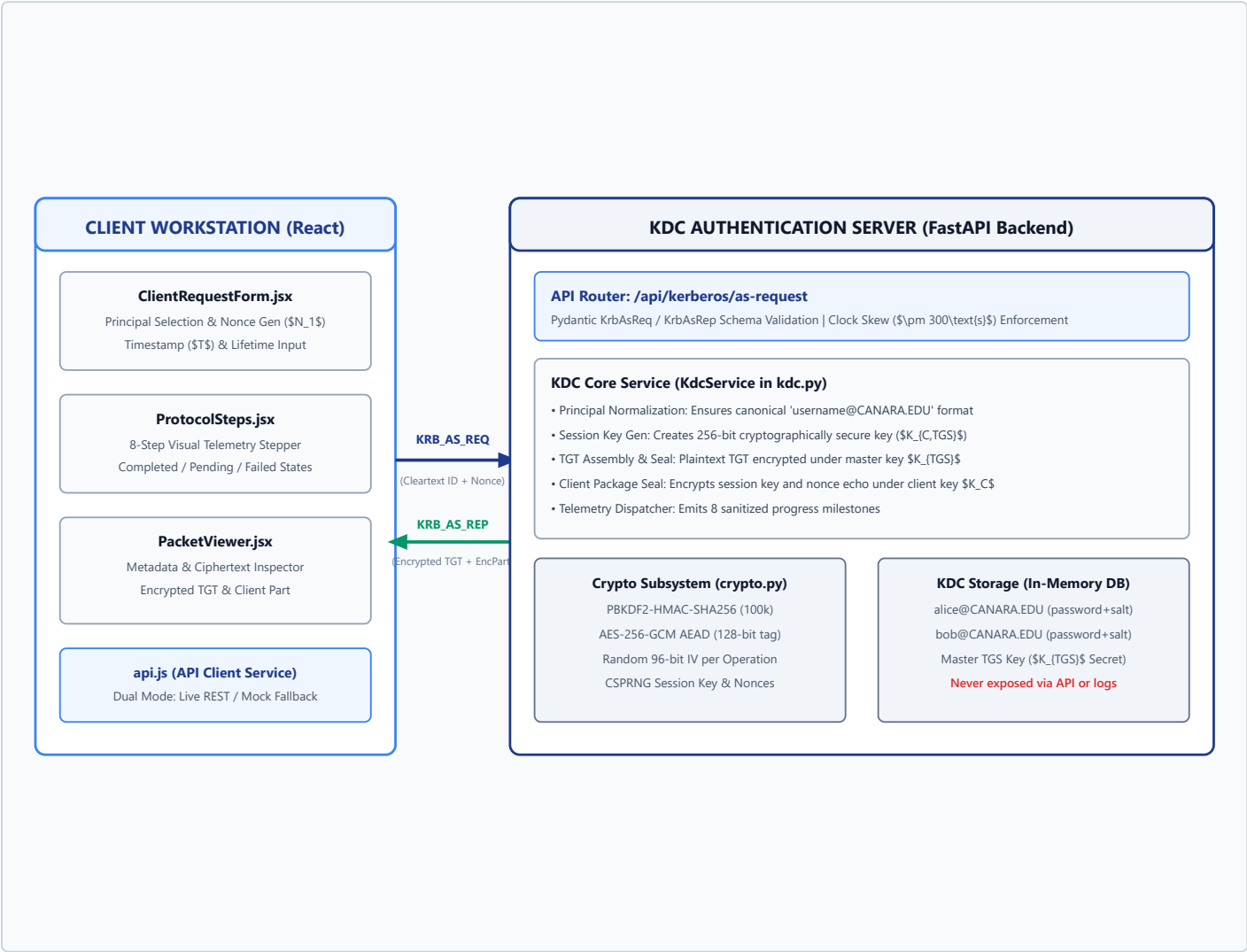
2

System Design & Architecture

2.1 Architectural Overview

The simulation implements a decoupled, full-stack client-server architecture designed around strict boundaries of cryptographic isolation. The client application models a user workstation requiring authentication. The server application acts as the KDC Authentication Server. Communication occurs over standard HTTP REST endpoints with strict JSON schemas. Sensitive cryptographic secrets (user master passwords, derived encryption keys, and unencrypted session keys) are strictly contained within their respective trust domains and never cross the wire.

2.2 System Architecture Diagram



2.3 Component Description & Trust Boundaries

Client Simulator: Encapsulates user input and request formulation. It has zero knowledge of server master keys or other users' credentials.

KDC Authentication Server: The authoritative entity containing user accounts and service keys. It generates ephemeral session keys and seals them into ticket structures.

Ticket Granting Service (TGS): Represented conceptually via the master service key \$K_{TGS}\$. The TGT issued by the AS can only be decrypted and accepted by the TGS in subsequent protocol phases.

3

Kerberos AS Exchange Workflow

3.1 Step-by-Step Execution Sequence

The implemented Authentication Server exchange faithfully models the 8 fundamental telemetry steps defined by the Kerberos V5 specification (RFC 4120), transitioning from client request submission to structured response delivery:

Step	Milestone Name	Technical Processing & Cryptographic Actions	Key Utilized	Security Goal
Step 1	Request received	Client constructs and transmits `KRB_AS_REQ` containing `cname`, `realm`, `sname`, `nonce` (\$N_1\$), timestamp (\$T\$), and requested lifetime. No password or hash is sent.	None (Cleartext)	Identity Assertion
Step 2	Principal validated	KDC verifies realm equals `CANARA.EDU`, service equals `krbtgt/CANARA.EDU`, and evaluates clock freshness: $ T_{\text{server}} - T_{\text{client}} \leq 300 \text{ seconds}$.	None	Replay & Bounds Defense
Step 3	KDC lookup completed	Server searches in-memory store. If user exists, server retrieves credential and derives client key \$K_C\$ via PBKDF2. If absent, exchange halts with `KDC_ERR_C_PRINCIPAL_UNKNOWN`.	Derived \$K_C\$ (PBKDF2)	Identity Verification
Step 4	Session key generated	KDC generates a fresh 256-bit ephemeral symmetric session key (\$K_{C,TGS}\$) using a cryptographically secure pseudo-random number generator (secrets.token_bytes(32)).	None (Ephemeral)	Key Freshness
Step 5	TGT constructed	Plaintext Ticket Granting Ticket is assembled with fields: `cname`, `sname`, `issue_time`, `expire_time`, and the embedded session key \$K_{C,TGS}\$.	Plaintext Struct	Ticket Formatting
Step 6	TGT encrypted	TGT structure is encrypted using KDC master service key \$K_{TGS}\$ via AES-256-GCM with a fresh 96-bit IV. Produces opaque ciphertext with 128-bit integrity tag.	Master Key \$K_{TGS}\$	TGT Tamper-Proofing
Step 7	Client response encrypted	Client response package containing \$K_{C,TGS}\$, nonce echo (\$N_1\$), expiry, and realm is encrypted using derived client key \$K_C\$ via AES-256-GCM with fresh 96-bit IV.	Client Key \$K_C\$	Confidential Key Delivery
Step 8	AS response generated	KDC packages encrypted TGT and encrypted client part into final `KRB_AS_REP` structure, appends sanitized telemetry steps, and returns HTTP 200 to client.	Structured Transport	Response Dispatch

3.2 Error Handling & Stepped Failure Progression

A critical pedagogical feature of our simulation is **stepped failure progression**. When an unauthorized principal (e.g., `charlie@CANARA.EDU`) initiates an AS request, the KDC does not produce a generic crash. Instead:

- **Steps 1 & 2** successfully evaluate to **COMPLETED** because the packet syntax and realm are well-formed.
- **Step 3** flags as **FAILED** with RFC error code `KDC\_ERR\_C\_PRINCIPAL\_UNKNOWN` (HTTP 404).
- **Steps 4 through 8** remain strictly in **PENDING** state, visually and logically proving that no session key was minted and no ticket was issued.
- The frontend Packet Inspector immediately purges any previous successful tickets, preventing stale cache trust.

4 Cryptographic Implementation

4.1 Key Derivation Function: PBKDF2-HMAC-SHA256

Standard passwords are vulnerable to dictionary search. In Kerberos, passwords are converted into symmetric cryptographic keys using a string-to-key function. In our simulation, the KDC implements PBKDF2-HMAC-SHA256 according to NIST SP 800-132:

Key Derivation Specification:

$$\text{Key} = \text{PBKDF2}(\text{PRF} = \text{HMAC-SHA256}, \text{Password} = \text{UserPassword}, \text{Salt} = \text{Realm} \parallel \text{Username}, c = 100,000, \text{dkLen} = 32 \text{ bytes})$$

Security Benefit: The 100,000 iterations introduce a deliberate computational cost (work factor) of several milliseconds per derivation. While imperceptible during legitimate authentication, it renders offline brute-force and rainbow table attacks computationally intractable for eavesdroppers.

4.2 Authenticated Symmetric Encryption: AES-256-GCM

Older Kerberos specifications (RFC 1510) relied on DES or CBC-mode ciphers requiring separate HMACs. Our implementation utilizes **AES-256-GCM (Galois/Counter Mode)**, an Authenticated Encryption with Associated Data (AEAD) cipher specified in NIST SP 800-38D.

- Confidentiality:** Counter (CTR) mode encrypts the ticket payload and client package with 256-bit security.
- Integrity & Authenticity:** GCM computes a 128-bit GHASH authentication tag over the ciphertext. If an attacker tampers with even a single bit of the encrypted TGT or client payload, decryption fails immediately with an 'InvalidTag' exception.
- Unique IV / Nonce:** Every invocation of 'encrypt_aes_gcm()' generates a fresh 96-bit (12-byte) cryptographically secure random nonce via 'secrets.token_bytes(12)'. This ensures identical plaintext blocks produce distinct ciphertexts, completely preventing keystream reuse attacks.

4.3 Nonce Management vs. Multi-Factor Replay Defense

A critical distinction emphasized in this course project is the precise role of nonces in protocol security:

- Client Nonce (\$N_1\$):** A 32-bit cryptographically secure random integer generated by the client ('generate_nonce()'). Its primary purpose is **request-response correlation**—guaranteeing that the 'KRB_AS_REP' received matches the immediate 'KRB_AS_REQ' submitted.
- Comprehensive Replay Protection:** In Kerberos, replay defense does **not** rely on the nonce alone. It is achieved through the coordinated operation of four distinct mechanisms: (1) Client nonces, (2) Server-enforced UTC timestamps (\$T_{server} - T_{client} \le 300\$), (3) Ticket validity lifetimes (\$300 \le L \le 86400\$), and (4) Authenticator timestamps in subsequent phases.

4.4 Cryptographic Parameters Summary Table

Cryptographic Primitive	Algorithm / Standard	Parameters / Configuration	Operational Purpose
Key Derivation (KDF)	PBKDF2-HMAC-SHA256	100,000 iterations, 32-byte output, Salt: 'Realm + Username'	Derive client secret key \$K_C\$ from user credentials
Ticket Encryption	AES-256-GCM (AEAD)	256-bit key (\$K_{TGS}\$), 96-bit IV, 128-bit tag	Seal TGT so client cannot read or modify contents
Client Package Encryption	AES-256-GCM (AEAD)	256-bit key (\$K_C\$), 96-bit IV, 128-bit tag	Deliver session key \$K_{[C,TGS]}\$ and nonce echo safely
Session Key Generation	CSPRNG ('secrets')	256-bit (32 bytes) cryptographically random, base64	Ephemeral symmetric key for Client-TGS communication
Clock Skew Tolerance	UTC Comparison	\$\pm 300\$ seconds (\$\pm 5\$ minutes) maximum offset	Reject stale or replayed authentication requests

5 Implementation & Core Code

5.1 Request Validation & Clock-Skew Check (`schemas/kerberos.py`)

```
class KrbAsReq(BaseModel):
    cname: str; realm: str; sname: str; nonce: int; timestamp: str; lifetime: int
    # Enforces strict 300-second clock skew tolerance at the schema boundary
    @field_validator("timestamp")
    def validate_freshness(cls, v: str) -> str:
        req_time = datetime.fromisoformat(v)
        skew = abs((datetime.now(timezone.utc) - req_time).total_seconds())
        if skew > 300.0:
            raise ValueError(f"Clock skew exceeded: {skew:.1f}s > 300s tolerance")
        return v
```

Explanation: Utilizes Pydantic V2 to enforce clock skew freshness before requests reach the core service, immediately rejecting replayed or desynchronized requests.

5.2 Principal Lookup & Key Derivation (`services/kdc.py`)

```
def get_client_key(self, cname: str) -> bytes:
    normalized = self.normalize_principal(cname)
    if normalized not in self._user_db:
        raise KerberosPrincipalNotFoundError(f"Principal '{cname}' not in KDC database.")
    user = self._user_db[normalized]
    return derive_key_from_password(password=user["password"], salt=user["salt"])
```

Explanation: Normalizes principal strings, searches the KDC database, raises explicit domain errors for unknown users, and executes PBKDF2 key derivation using the user's password and salt.

5.3 Authenticated Encryption with Fresh IV (`core/crypto.py`)

```
def encrypt_aes_gcm(key: bytes, plaintext_data: Any) -> Tuple[str, str]:
    raw_bytes = json.dumps(plaintext_data).encode("utf-8")
    # Fresh, unique 12-byte random IV for every single encryption
    nonce = secrets.token_bytes(12)
    aesgcm = AESGCM(key)
    ciphertext_with_tag = aesgcm.encrypt(nonce, raw_bytes, associated_data=None)
    return base64.b64encode(ciphertext_with_tag).decode(), base64.b64encode(nonce).decode()
```

Explanation: Encrypts payload dictionaries using AES-256-GCM. Generates a fresh 96-bit nonce via CSPRNG for every call, preventing keystream reuse and attaching a 128-bit authentication tag.

5.4 Core AS Request Processing Pipeline (`services/kdc.py`)

```
# Ephemeral session key generated via CSPRNG (256-bit)
session_key = generate_session_key()
# Step 6: Encrypt TGT under master K_TGS key (client cannot decrypt)
tgt_cipher, tgt_iv = encrypt_aes_gcm(self._k_tgs, tgt_content)
# Step 7: Encrypt client response under derived K_C key (contains session key & nonce)
client_cipher, client_iv = encrypt_aes_gcm(client_key, client_enc_content)
return KrbAsRep(ticket=EncryptedTicketSchema(cipher_text=tgt_cipher, nonce=tgt_iv),
                enc_part=EncryptedClientPartSchema(cipher_text=client_cipher, nonce=client_iv))
```

Explanation: Demonstrates the central Kerberos duality: the session key is sealed inside the TGT (for TGS) and also sealed inside the client package (for client), allowing shared secret establishment without network exposure.

6 Results & Test Verification

6.1 Operational Demonstration Scenarios

The implementation was verified on `http://localhost:5173/` integrated with the active FastAPI server (`http://127.0.0.1:8000`). Four distinct verification scenarios were confirmed:

- **Valid User Authentication (Alice & Bob):** Successful request processing with HTTP 200. All 8 telemetry steps display green `COMPLETED` checks. Packet inspector renders `alice@CANARA.EDU` cleanly with encrypted TGT and client ticket. Bob's authentication generates a distinct 256-bit session key.
- **Unauthorized User Rejection (Charlie):** When `charlie@CANARA.EDU` is submitted, Steps 1–2 pass syntax check, Step 3 flags as red `FAILED`, and Steps 4–8 remain grey `PENDING`. Returns `KDC\_ERR\_C\_PRINCIPAL\_UNKNOWN` (HTTP 404). Stale ticket data is completely cleared.
- **Clock Skew Defense:** Requests with timestamp offset > 300 seconds rejected with `KRB\_AP\_ERR\_SKEW` (HTTP 400).

6.2 Automated Test Suite Results (100% Passing — 21 / 21 Tests)

Verification was conducted using automated test runners across both frontend and backend suites:

Suite	Test Target	Test Method / Assertion	Result
Backend	Valid AS Exchange	Verifies HTTP 200, ciphertext generation, and 8 telemetry steps	PASSED
Backend	Unknown Principal	Asserts HTTP 404 and `KDC_ERR_C_PRINCIPAL_UNKNOWN` for unregistered user	PASSED
Backend	Unsupported Service	Rejects service principals other than `krbtgt/CANARA.EDU`	PASSED
Backend	Stale Timestamp	Rejects timestamps older than 300 seconds with clock skew error	PASSED
Backend	Schema Validation	Rejects negative nonces, invalid lifetimes, and malformed emails	PASSED
Backend	Decryption Integrity	Verifies client key successfully decrypts client part; matches session key	PASSED
Backend	Tamper Detection	Verifies incorrect key or tampered ciphertext triggers AES `InvalidTag`	PASSED
Backend	Session Key Uniqueness	Verifies subsequent requests receive distinct session keys and IVs	PASSED
Backend	Secret Leakage Check	Scans raw JSON responses to confirm zero passwords/salts leaked	PASSED
Backend	Health Check API	Confirms `/api/health` backward compatibility and course metadata	PASSED
Frontend	Component Rendering (6)	Validates Header, Form, 8-step Timeline, Formatter, and Status alerts	PASSED
Frontend	API Client Service (5)	Verifies health check, principal fetching, AS exchange, and offline mock	PASSED

6.3 Proof of Implementation (Live Application Screenshots)

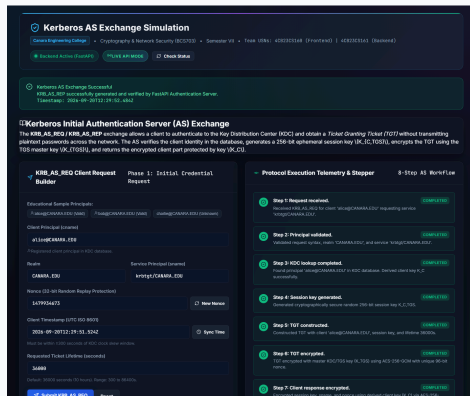


Figure 6.1: Alice Successful AS Exchange (All 8 Steps Completed)

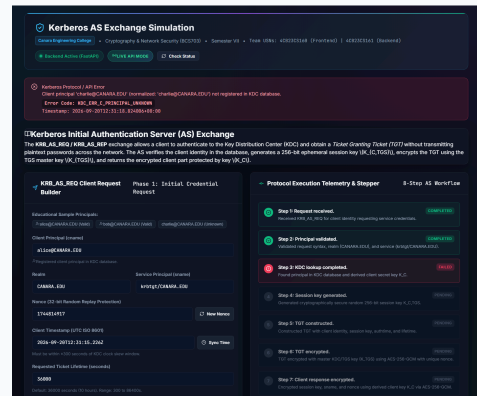


Figure 6.2: Charlie Unknown Principal Rejection (Step 3 Failed)

7 Conclusion & Future Scope

7.1 Project Outcomes & Core Achievements

The Course Project successfully conceptualized, engineered, and verified a transparent, educational full-stack simulation of the Kerberos Authentication Server (AS) Exchange. The system bridges the pedagogical gap between theoretical cryptographic concepts and practical software engineering, providing students and evaluators with complete visual and technical transparency into initial Single Sign-On authentication mechanics.

7.2 Demonstrated Security Principles

- **Zero Password Transmission:** Proved that authenticating to a trusted KDC requires no transmission of passwords or password hashes across the network.
- **Work-Factor Resistance:** Implemented PBKDF2-HMAC-SHA256 (100,000 iterations) with salted inputs to neutralize offline dictionary attacks.
- **Tamper-Evident Encryption:** Applied AES-256-GCM authenticated encryption to guarantee that modified tickets raise explicit exceptions (`InvalidTag`).
- **Multi-Factor Replay Defense:** Combined client nonces (`$N_1$`) for request correlation with strict UTC timestamp windows (`$\le 300\text{s}$`).
- **Strict Error Isolation:** Handled invalid principals without disclosing cryptographic secrets or emitting false success tickets.

7.3 Educational Scope Limitations

This project is an **educational simulation** developed for Cryptography & Network Security (BCS703). It utilizes modern RESTful JSON over HTTP instead of legacy binary ASN.1 DER encodings over UDP port 88. User accounts are maintained in memory for demonstration purposes. While cryptographically sound for classroom analysis, it is not intended as an enterprise production replacement for Active Directory or MIT Kerberos.

7.4 Future Scope & Development Roadmap

- **Phase 4 — Ticket Granting Server (TGS) Exchange:** Implement ``KRB_TGS_REQ`` and ``KRB_TGS_REP`` message handlers where the client presents its TGT and an Authenticator to request end-service tickets.
- **Phase 5 — Client/Server Application (AP) Exchange:** Implement mutual authentication (``KRB_AP_REQ`` / ``KRB_AP_REP``) between client and target network services (e.g., HTTP / File servers).
- **Public Key Cryptography (PKINIT):** Extend the initial AS exchange with RFC 4556 (PKINIT) to support X.509 digital certificates and asymmetric key pairs.

7.5 Conclusion

The project fulfills all requirements established by the syllabus for Cryptography & Network Security (BCS703). By integrating industry-standard cryptography (FastAPI, AES-256-GCM, PBKDF2) with an intuitive reactive frontend, the simulation provides an exemplary pedagogical instrument that demystifies complex network security protocols.

7.6 References

1. Neuman, C., Yu, T., Hartman, S., & Raeburn, K. (2005). *The Kerberos Network Authentication Service (V5)*. RFC 4120, Internet Engineering Task Force (IETF).
2. National Institute of Standards and Technology (NIST). (2010). *Recommendation for Password-Based Key Derivation (PBKDF2)*. NIST Special Publication 800-132.
3. Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*. NIST Special Publication 800-38D.
4. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice* (7th ed.). Pearson Education.
5. FastAPI & Pydantic Documentation. (2024). *Asynchronous REST Framework and Data Validation for Python*. <https://fastapi.tiangolo.com>